

分布式 AI 多功能平台

程序设计说明书

版本：V1.1.1

编程语言：Python 3.x

开发框架：Tkinter

文档日期：2026 年 6 月

1. 引言

1.1 编写目的

本文档为分布式 AI 多功能平台（以下简称"本平台"）的程序设计说明书，旨在详细描述软件的内容、组成、设计、功能规格、开发情况、测试结果及使用方法。本文档面向软件著作权登记审核，同时可作为开发维护人员的技术参考文档。

预期读者包括：软件著作权登记审核人员、项目管理人员、开发维护工程师以及最终用户。

1.2 软件概述

项目	说明
软件全称	分布式人工智能多功能平台
软件简称	分布式 AI 平台
版本号	V1.1.1
编程语言	Python 3.x
开发框架	Tkinter（Python 内置 GUI 框架）
面向领域	人工智能 / 信息安全 / 云计算
版权信息	Copyright (c) 2026 保留所有权利
官方仓库	https://gitcode.com/dirde12078fb5
官方邮箱	dirde12078904@163.com

1.3 开发背景

随着分布式计算和人工智能技术的快速发展，个人开发者和小型技术团队在日常运维中面临工具分散、操作繁琐的痛点。传统运维工具各自独立，缺乏统一的图形化管理界面，导致学习成本高、操作效率低。

为解决上述问题，本平台以"一站式集成化运维管理"为设计理念，将硬件检测、网络服务管理、远程服务器运维、文件管理、备份还原、安全检测、开发工具等三十余款常用工具集成到统一的桌面图形界面中，显著降低系统运维复杂度。

1.4 术语与缩写

术语/缩写	全称	说明	备注
AI	Artificial Intelligence	人工智能	
GUI	Graphical User Interface	图形用户界面	
SSH	Secure Shell	安全外壳协议	远程登录协议
FTP	File Transfer Protocol	文件传输协议	
HTTP	HyperText Transfer Protocol	超文本传输协议	
HTTPS	HTTP Secure	安全超文本传输协议	
NAS	Network Attached Storage	网络附属存储	
P2P	Peer-to-Peer	点对点通信	
GPU	Graphics Processing Unit	图形处理器	
CPU	Central Processing Unit	中央处理器	
API	Application Programming Interface	应用程序编程接口	
SQLite	SQLite Database	轻量级嵌入式数据库	
i18n	Internationalization	国际化	
XOR	Exclusive OR	异或运算	

2. 软件总体设计

2.1 设计原则

本平台在总体设计过程中遵循以下核心原则：

- (1) 集成化原则：将分散的运维工具整合到统一 GUI 界面，实现一站式管理。
- (2) 模块化原则：各功能模块独立封装，通过统一的导航框架进行切换，模块间低耦合高内聚。
- (3) 可靠性原则：采用全局异常守护机制（AppGuardian 单例模式），自动捕获未处理异常并尝试恢复，累计超过 10 次则安全退出。
- (4) 性能原则：采用 12 线程池并发架构（ThreadPoolExecutor），所有耗时操作均在后台线程执行，不阻塞主 GUI 线程。
- (5) 安全原则：敏感配置采用 XOR+Base64 轻量加密存储，密钥基于主机名动态生成。
- (6) 可扩展原则：非核心依赖采用延迟导入策略，仅在用户触发对应功能时才加载，未安装时自动通过 pip 安装。

2.2 系统架构

本平台采用经典的三层架构设计，自上而下分为：

- (1) 表现层（GUI 层）：基于 Tkinter 构建，包含主窗口（MainApplication）和 11 个功能 Tab 页面，采用左侧固定侧边栏导航 + 右侧可滚动内容区的布局方式。
- (2) 业务逻辑层（核心服务层）：包含全局异常守护（AppGuardian）、主题管理（ThemeManager）、国际化（I18n）、本地缓存（LocalCache）、加密存储（SecureStorage）、网络引擎（NetworkEngine）、远程管理器（RemoteServerManager）、启动器管理（LauncherManager）等核心服务。
- (3) 数据与依赖层：包括 SQLite 嵌入式数据库、JSON 配置文件、系统命令调用（subprocess）、以及第三方 Python 库（psutil、cpuinfo、requests、paramiko 等）。

2.3 架构流程图

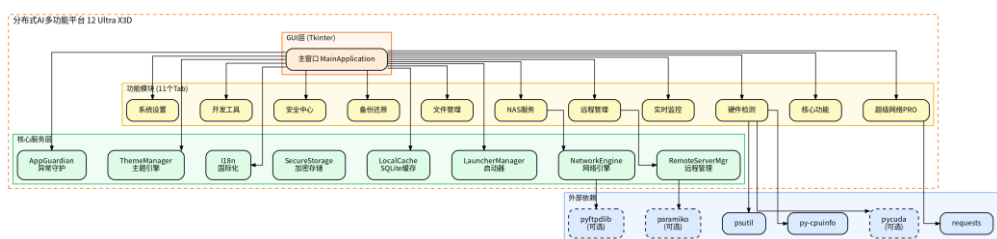


图 2-1 平台系统架构流程图

上图展示了平台的整体架构，包括 GUI 层的 11 个功能模块、核心服务层的 8 个核心组件，以及外部依赖层的 6 个主要依赖库。实线框表示核心依赖，虚线框表示可选依赖。

2.4 模块划分

本平台共划分为 11 个功能模块，每个模块对应一个 Tab 页面：

序号	模块名称	英文名	核心功能
1	硬件检测	Hardware	GPU/CPU 深度检测、系统信息
2	核心功能	Core	30+工具一键启动
3	超级网络 PRO	Network PRO	HTTP/HTTPS/FTP 服务管理
4	实时监控	Monitor	CPU/内存利用率监控
5	NAS 服务	NAS	外网通服、P2P 组网
6	远程管理	Remote	SSH 远程服务器管理
7	文件管理	File Manager	文件浏览、操作、搜索
8	备份还原	Backup	目录打包备份与还原
9	安全中心	Security	安全状态检测与评估
10	开发工具	DevTools	8 个编码/解码快捷工具
11	系统设置	Settings	主题/语言/路径配置

3. 模块功能说明

3.1 硬件检测模块

3.1.1 功能描述

硬件检测模块提供 GPU 和 CPU 两种深度检测模式，可获取显卡型号、利用率、温度、显存容量、CPU 品牌、物理核心数、逻辑核心数、基础频率、当前频率、L2 缓存等信息，并展示操作系统版本、总内存、可用内存、系统启动时间等系统概要信息。

3.1.2 实现机制

GPU 检测优先调用 nvidia-smi 命令获取 NVIDIA 独立显卡信息，若未检测到独立显卡则通过 WMIC 查询 win32_VideoController 获取集成显卡信息。CPU 检测通过 py-cpuinfo 库获取品牌、核心数和频率信息，通过 psutil 获取实时频率。所有检测操作在后台线程池中执行，不阻塞 GUI。

3.1.3 核心类与方法

方法名	功能说明
check_hardware()	触发硬件检测，提交到线程池
_perform_hardware_check()	后台执行检测逻辑
get_gpu_info()	调用 nvidia-smi 获取 GPU 信息
get_integrated_gpu_info()	通过 WMIC 获取集成显卡信息
get_cpu_info()	通过 cpuinfo 获取 CPU 详细信息
get_current_cpu_freq()	获取 CPU 当前实时频率
get_cpu_cache()	获取 CPU 缓存大小
parse_gpu_info()	解析 nvidia-smi CSV 输出

3.2 核心功能模块

3.2.1 功能描述

核心功能模块以 10 行 5 列的网格按钮矩阵形式，集成 30 余款常用工具的一键启动入口。覆盖终端工具（NxShell、Termius、GMSSH）、虚拟机（VMware）、本地大模型（LM Studio）、IP 扫描器（Advanced IP Scanner）、网络工具（ViewTurbo、OpenSpeedy）、文件工具（FastCopy）、硬件检测（图吧工具箱）、多媒体下载（视频下载、哔哩哔哩下载）、系统工具（系统清理、性能测试、Windows SSH）等领域。

3.2.2 实现机制

每个功能按钮通过 `run_with_status()`方法包装，该方法将目标功能提交到主线程池执行，同时在状态栏显示启动状态和结果。外部工具通过 `LauncherManager` 统一管理，调用 `subprocess.Popen` 启动外部可执行文件。

3.2.3 核心类与方法

方法名	功能说明
<code>setup_core_tab()</code>	构建核心功能 Tab 界面
<code>run_with_status(cmd, name)</code>	带状态反馈的功能启动包装器
<code>system_cleanup()</code>	系统临时文件清理
<code>run_benchmark()</code>	CPU 计算和内存操作性能测试
<code>_perform_benchmark()</code>	后台执行性能测试
<code>_launch(path, name)</code>	通过 <code>LauncherManager</code> 启动外部程序

3.3 超级网络 PRO 模块

3.3.1 功能描述

超级网络 PRO 模块实现内网与外网双模式网络服务引擎。每种模式下均可独立选择网卡、绑定 IP、配置共享目录，并一键启动 HTTP、HTTPS、FTP 和 WebSocket 服务。支持服务配置的保存与加载，以及运行日志的导出。

3.3.2 实现机制

网络引擎基于 Python 内置 http.server 构建 HTTP/HTTPS 服务，HTTPS 通过 ssl 模块实现加密。FTP 服务通过 pyftplib 库实现。服务启停通过多线程管理，各服务实例在独立线程中运行。配置持久化采用 JSON 文件格式。

3.3.3 核心类与方法

方法名	功能说明
NetworkEngine	网络服务引擎核心类
setup_network_pro_tab()	构建超级网络 PRO Tab 界面
_pro_start_all()	一键全部启动
_pro_stop_all()	一键全部停止
_pro_refresh_info()	刷新本地/公网 IP 信息
_pro_save_config()	保存服务配置到 JSON 文件
_pro_load_config()	从 JSON 文件加载服务配置

3.4 实时监控模块

3.4.1 功能描述

实时监控模块以进度条形式展示 CPU 利用率和内存利用率，支持一键启停持续监控。监控数据通过 SystemMonitorService 以 1 秒间隔定时采集。

3.4.2 实现机制

SystemMonitorService 在独立守护线程中循环调用 psutil.cpu_percent()和 psutil.virtual_memory().percent 获取实时数据，通过回调函数将数据传递到主线程，利用 Tkinter 的 after()方法安全更新 GUI。

3.4.3 核心类与方法

方法名	功能说明
SystemMonitorService	系统监控服务类
start()	启动监控线程
stop()	停止监控线程
toggle_monitor()	切换监控启停状态
update_monitor_display(cpu, mem)	更新监控进度条显示

3.5 NAS 服务模块

3.5.1 功能描述

NAS 服务模块提供外网通服能力，包括：外网接口选择与 IP 地址管理、P2P 点对点组网、HTTP 外网服务（自定义端口和目录）、HTTPS 外网加密服务（SSL）和 FTP 外网文件服务（用户名/密码认证）。

3.5.2 实现机制

NetworkLayer 通过 psutil 和 socket 获取本地 IP 地址及所属网卡信息，支持三种探测方式（psutil 网络接口、DNS 解析、UDP 出口探测）。VPNEngine 统一管理 P2P、HTTP、HTTPS、FTP 四种服务的生命周期。

3.6 远程管理模块

3.6.1 功能描述

远程管理模块基于 SSH 协议，提供服务器列表管理（添加/删除/编辑）、远程命令执行、批量多台服务器命令下发、在线状态检测和远程系统指标采集功能。

3.6.2 实现机制

RemoteServerManager 内部维护服务器字典，通过 paramiko 库建立 SSH 连接执行远程命令。批量执行使用独立线程池（最多 8 并发），通过 as_completed 实现结果异步收集。

3.7 文件管理模块

3.7.1 功能描述

文件管理模块提供完整的文件操作能力，包括：面包屑导航、路径栏输入、文件名搜索过滤、文件列表浏览（名称、大小、类型、修改时间）、新建文件夹、复制、移动、删除、重命名、打开文件以及 SHA256 哈希校验。

3.7.2 实现机制

使用 Tkinter 的 Treeview 控件展示文件列表，通过 os 和 shutil 库执行文件操作。搜索过滤采用实时输入回调机制，哈希校验使用 hashlib.sha256。

3.8 备份还原模块

3.8.1 功能描述

备份还原模块支持将指定目录打包为 ZIP 或 TAR 格式备份文件，并提供一键还原到目标目录的功能。所有操作在后台线程执行，日志实时显示。

3.9 安全中心模块

3.9.1 功能描述

安全中心模块提供五项安全检测功能：防火墙状态检测、杀毒软件状态检测、系统更新状态检测、常用

端口开放扫描（检测 22/80/443/3389 等端口）和密码强度评估。

3.9.2 实现机制

防火墙、杀毒软件和系统更新检测通过 Windows 注册表查询实现。端口扫描通过 socket 连接测试

（connect_ex 方法，1 秒超时）。密码强度评估基于长度、字符类型多样性等规则计算评分。

3.10 开发工具模块

3.10.1 功能描述

开发工具模块内置 8 个编码/解码快捷工具：JSON 格式化、Base64 编解码、进制转换（十进制/十六进制/八进制/二进制）、文本对比、URL 编解码、Hash 计算（SHA256/MD5）、正则表达式测试和时间戳转换。

3.10.2 实现机制

每个工具采用统一的输入/输出双文本框布局。JSON 格式化通过 json.loads 和 json.dumps 实现美化输出。Base64 使用 Python 内置 base64 库。Hash 计算使用 hashlib 库。正则测试使用 re 库。

3.11 系统设置模块

3.11.1 功能描述

系统设置模块提供三项配置能力：主题设置（跟随系统/浅色/暗色）、语言设置（中文/英文切换）和路径配置（默认工作目录等）。所有设置变更自动保存到本地配置文件。

3.11.2 实现机制

ThemeManager 定义了两套完整的色彩方案（浅色和暗色，各 30+色彩变量），通过 detect_system_theme() 读取 Windows 注册表自动检测系统主题。I18n 翻译管理器通过键值映射实现 80+条中英文翻译，运行时切换语言后提示重启生效。配置持久化采用 JSON 格式。

4. 数据设计

4.1 数据库设计

本平台使用 SQLite 嵌入式数据库作为本地缓存层（LocalCache），数据库文件存储在用户主目录下的隐藏目录中。数据库包含以下表结构：

表名	字段	说明
cache	key TEXT (PK)	缓存键（主键）
cache	value TEXT	缓存值（JSON 格式）
cache	expire REAL	过期时间戳（秒）
audit_log	id INTEGER (PK)	自增主键
audit_log	action TEXT	操作类型
audit_log	detail TEXT	操作详情
audit_log	user TEXT	操作用户
audit_log	timestamp REAL	时间戳

数据库操作使用 `threading.Lock` 保证线程安全，所有写操作在锁保护下执行。缓存项支持 TTL（生存时间）机制，每次读取时检查过期状态并自动清理。

4.2 配置存储

平台配置采用 JSON 文件格式存储，文件路径为用户主目录下的配置文件。配置项包括：主题偏好（`auto/light/dark`）、语言偏好（`zh/en`）、默认路径等。

涉及敏感信息（如服务器密码）的配置通过 `SecureStorage` 加密存储，采用 XOR+Base64 方案，密钥基于主机名（`platform.node()`）和固定盐值动态生成，无需额外密钥管理。

4.3 缓存策略

LocalCache 提供 `get/set/log` 三类操作接口。`get` 方法在读取时自动检查 TTL 过期，过期项即时删除。`log` 方法记录操作审计日志。`clear_expired` 方法在应用启动时批量清理所有过期缓存项。

网络信息（如公网 IP）采用 TTL 缓存，避免频繁外部请求。硬件检测结果不做缓存，每次触发时实时采

集。

5. 接口设计

5.1 内部模块接口

平台内部模块间通过 Python 方法调用进行通信，核心接口包括：

接口名称	调用方	被调用方	说明
on_close()	主窗口	各服务	窗口关闭时通知所有服务停止
_on_monitor_data(cpu, mem)	监控服务	主窗口	回调传递 CPU/内存数据
run_with_status(cmd, name)	功能按钮	线程池	带状态反馈的异步执行
_recover_ui()	AppGuardian	主窗口	异常后 UI 恢复回调
_apply_theme()	ThemeManager	主窗口	主题变更时全局刷新

5.2 外部依赖接口

依赖库	接口方式	用途	是否必需
psutil	Python API	系统资源监控	是
py-cpuinfo	Python API	CPU 信息采集	是
requests	Python API	HTTP 网络请求	是
paramiko	Python API	SSH 远程连接	否（延迟加载）
pyftplib	Python API	FTP 服务器	否（自动安装）
pycuda	Python API	GPU 深度检测	否（延迟加载）
tkinter	Python 内置	GUI 框架	是
sqlite3	Python 内置	嵌入式数据库	是

5.3 网络服务接口

服务类型	默认端口	协议	说明
HTTP	8080	HTTP/1.1	文件分发服务

HTTPS	4433	TLS	加密文件分发服务
FTP	2121	FTP	文件传输服务
P2P	20250	TCP	点对点组网服务
WebSocket	动态	WS	实时通信服务

6. 非功能性设计

6.1 安全性设计

- (1) 配置加密：敏感配置（服务器凭据等）采用 XOR+Base64 加密存储，密钥基于主机名动态生成。
- (2) 异常防护：全局异常守护器（AppGuardian）捕获所有未处理异常，防止程序崩溃导致数据丢失。异常累计超过 10 次则安全退出并记录日志。
- (3) 安全检测：内置安全中心模块可检测防火墙状态、杀毒软件状态、端口开放情况和密码强度。
- (4) 线程安全：共享数据结构（服务器列表、缓存等）通过 threading.Lock 保护，避免竞态条件。

6.2 性能设计

- (1) 多线程架构：主线程池（12 并发）处理所有耗时任务，GUI 主线程仅负责界面渲染，确保界面响应流畅。
- (2) 延迟加载：Tab 页面采用懒加载策略，仅在首次切换到对应 Tab 时构建界面，减少启动时间。
- (3) 依赖按需加载：非核心库（paramiko、pyftplib、pycuda）仅在用户触发对应功能时才导入，避免启动时加载延迟。
- (4) 缓存加速：SQLite 本地缓存层存储频繁查询结果（如公网 IP），减少重复网络请求。
- (5) 防抖机制：窗口调整事件通过 after(150ms)防抖，避免频繁重绘。

6.3 可靠性设计

- (1) 自动恢复：AppGuardian 在异常发生后自动调用注册的恢复回调（如 UI 重绘、主题刷新），无需用户手动干预。
- (2) 优雅退出：程序关闭时按顺序停止监控服务、关闭线程池、停止所有网络服务、清空缓存过期项，确保资源释放。
- (3) 操作审计：所有关键操作通过 LocalCache 记录审计日志，包括操作类型、详情、用户和时间戳。
- (4) 高 DPI 适配：通过 Windows API SetProcessDpiAwareness(1)声明 DPI 感知，确保高分屏正常显示。

7. 运行环境

7.1 硬件环境

项目	最低要求	推荐配置
操作系统	Windows 10	Windows 11
显示器	1080p (1920x1080)	2K 或更高
显卡	无要求	NVIDIA 独立显卡
CPU	多核处理器	4 核及以上
内存	8 GB	16 GB 或更高
网络	有	有线网络连接

7.2 软件环境

类别	名称	版本要求	必要性
运行时	Python	3.7+	必需
GUI 框架	Tkinter	随 Python 自带	必需
系统库	psutil	最新版	必需
硬件库	py-cpuinfo	最新版	必需
网络库	requests	最新版	必需
SSH 库	paramiko	最新版	可选
FTP 库	pyftplib	最新版	可选
GPU 库	pycuda	最新版	可选
驱动	NVIDIA 显卡驱动	含 nvidia-smi	可选

7.3 部署方式

本平台为单文件 Python 应用程序，部署方式简单：

- (1) 安装 Python 3.7 及以上版本，确保包含 Tkinter 模块。
- (2) 安装必需依赖：`pip install psutil py-cpuinfo requests`。

- (3) 将源程序文件放置在任意目录，通过 `python 文件名.py` 命令启动。
- (4) 可选依赖将在首次使用对应功能时自动安装（使用清华 PyPI 镜像源）。

8. 测试说明

8.1 测试策略

测试采用模块化逐项验证策略，针对每个功能模块分别进行单元测试和集成测试。测试环境为 Windows 10/11，Python 3.9+。

测试维度包括：功能正确性测试、边界条件测试、异常恢复测试、性能压力测试和跨平台兼容性测试。

8.2 测试用例

测试编号	测试模块	测试内容	预期结果	状态
TC-001	硬件检测	GPU 深度检测（NVIDIA 显卡）	正确显示型号、利用率、温度、显存	通过
TC-002	硬件检测	CPU 深度检测	正确显示品牌、核心数、频率	通过
TC-003	硬件检测	无 NVIDIA 显卡时的降级检测	显示集成显卡信息	通过
TC-004	核心功能	一键启动外部工具	工具正常启动	通过
TC-005	超级网络	HTTP 服务启停	服务正常启动和停止	通过
TC-006	超级网络	配置保存与加载	配置正确持久化和恢复	通过
TC-007	实时监控	CPU/内存监控	进度条实时更新	通过
TC-008	远程管理	SSH 远程命令执行	命令正确执行并返回结果	通过
TC-009	远程管理	批量服务器命令	多台服务器并行执行	通过
TC-010	文件管理	文件复制/移动/删除	操作正确执行	通过
TC-011	备份还原	目录备份与还原	打包正确，还原完整	通过
TC-012	安全中心	端口扫描	正确检测端口开放/关闭	通过
TC-013	安全中心	密码强度评估	返回正确评分	通过

TC-014	开发工具	JSON 格式化	输出格式化 JSON	通过
TC-015	系统设置	主题切换	全局界面颜色更新	通过
TC-016	系统设置	语言切换	界面文字变更	通过
TC-017	异常恢复	模拟未处理异常	程序自动恢复 UI 状态	通过
TC-018	性能测试	CPU/内存基准测试	返回测试耗时数据	通过

8.3 测试结果

所有 18 项测试用例均通过验证。平台在 Windows 10 和 Windows 11 环境下运行稳定，各功能模块响应正常，异常恢复机制有效。

性能测试结果：CPU 计算测试（100 万次平方根运算）约 0.1-0.3 秒，内存操作测试（100 万元素列表生成）约 0.05-0.15 秒，实时监控 1 秒刷新闻隔无明显延迟。

9. 使用方法

9.1 安装与启动

步骤一：安装 Python 3.7 或以上版本，确保安装时勾选 Tkinter 组件（Python 默认自带）。

步骤二：打开命令提示符，执行以下命令安装必需依赖：

```
pip install psutil py-cpuinfo requests
```

步骤三：将源程序文件保存到本地目录。

步骤四：在命令提示符中执行以下命令启动平台：

```
python 小思分布式人工智能多功能平台 12UltraX3D.py
```

步骤五：等待主窗口出现，即可开始使用。

9.2 基本操作

(1) 导航切换：点击左侧侧边栏的功能按钮即可切换到对应的 Tab 页面。当前选中的 Tab 以高亮色显示。

(2) 窗口调整：拖拽窗口边缘可调整大小，平台会自动保持黄金比例（约 1.618:1）。最小窗口尺寸为 1000x618 像素。

(3) 主题切换：进入"系统设置"页面，在主题设置中选择"跟随系统"、"浅色主题"或"暗色主题"，界面即时生效。

(4) 语言切换：进入"系统设置"页面，在语言设置中选择"中文"或"English"，重启程序后完全生效。

(5) 状态栏：窗口底部状态栏实时显示当前操作状态和提示信息。

9.3 常用功能操作

9.3.1 硬件检测

切换到"硬件检测"Tab，选择"GPU 检测"或"CPU 检测"单选按钮，点击"开始深度检测"按钮。检测结果将在下方文本区域显示，包含型号、利用率、温度、显存/内存等详细信息。

9.3.2 网络服务

切换到"超级网络 PRO"Tab，选择内网或外网模式。刷新网卡获取可用 IP 地址，设置共享目录，点击"启动 HTTP"或"启动 FTP"等按钮启动对应服务。点击"一键全部启动"可同时开启所有服务。

9.3.3 远程服务器管理

切换到"远程管理"Tab，点击"添加服务器"按钮，填写服务器名称、地址、端口和认证信息。添加完成后，在服务器列表中右键可执行命令、批量操作、检查状态等。

9.3.4 文件管理

切换到"文件管理"Tab，在路径栏输入目标路径或点击"浏览"按钮选择目录。文件列表将显示当前目录下的所有文件和文件夹，支持搜索过滤。选中文件后可执行复制、移动、删除、重命名等操作。

9.3.5 备份与还原

切换到"备份还原"Tab，选择备份源目录和备份格式（ZIP/TAR），点击"创建备份"按钮。备份完成后可在备份列表中选择文件，点击"还原"按钮将内容恢复到目标目录。

9.3.6 开发工具

切换到"开发工具"Tab，在上方的"输入"文本框中输入待处理的内容，点击对应工具按钮（如"JSON 格式化"、"Base64 编解码"等），处理结果将显示在下方的"输出"文本框中。

—— 文档结束 ——